

Driving School Enrollment & Test Management System

Java Programming | Case Study 90
Java Swing Desktop Application

Author	Kartik Pravin Wagh
Program	B.Tech Computer Science & Engineering (2025-29), ITM Skills University, School of FutureTech
Subject	Java Programming, Semester III
Case Study	Case Study 90 - Driving School Enrollment & Test Management System
Source file	DrivingSchoolApp.java
Repository	github.com/kartikwagh21/Java_Major_Project

A Java Swing desktop application that manages student enrollment, tracks theory and practical training sessions, automatically evaluates test eligibility, and schedules driving tests for two-wheeler and four-wheeler courses.

Table of Contents

1. Introduction and Problem Statement
2. Objectives and Expected Modules
3. Features
4. Java Concepts Used
5. Software Design Overview
6. Class Descriptions
7. Data Structures
8. Business Rules
9. Exception Handling and Validation
10. Graphical User Interface
11. Module Walkthrough
12. Sample Data
13. How to Compile and Run
14. Test Scenarios and Expected Results
15. Output Screenshots
16. Limitations and Future Enhancements
17. Conclusion

1. Introduction and Problem Statement

A driving school needs a system to manage student enrollment, track theory and practical training sessions, and schedule driving tests across two-wheeler and four-wheeler courses. The system should flag students who are not yet eligible for their final test.

This project solves the problem with a single Java Swing desktop application, **DrivingSchoolApp**. It stores all records in memory using Java collections, applies the school's course rules automatically, and presents everything through a dark-themed graphical interface with four views: Student Enrollment, Training Sessions, Test Scheduling and Search & Analytics.

2. Objectives and Expected Modules

Objectives

- Maintain student and course enrollment information.
- Track theory and practical session attendance.
- Schedule driving tests.
- Determine test eligibility based on completed sessions.
- Search and sort enrollment records.
- Provide a driving school management GUI.

Expected modules and where they are implemented

Module	Implementation in the application
Student Enrollment	Students view: add, update, delete (createStudentsView, addStudent, updateStudent, deleteStudent)
Session Scheduling	Sessions view: log and edit THEORY / PRACTICAL sessions
Attendance Tracking	Session status COMPLETED / SCHEDULED / ABSENT stored per session
Eligibility Determination	checkEligibility() and previewEligibility() on the Tests view
Test Scheduling	scheduleTest() with eligibility gate and date validation
Enrollment Search	Search button on the Search & Analytics view (ID, name or course track)
Training Reports	TreeMap ranking, tests sorted by date, full school audit report

3. Features

- **Student enrollment (CRUD):** register students with Student ID, name, phone number, course track and fee paid; update details; delete a student together with all of that student's sessions and tests. The table shows fee status as PAID or DUE with the pending amount.
- **Session scheduling and attendance:** log THEORY and PRACTICAL sessions with a date and a status (COMPLETED, SCHEDULED, ABSENT); update or delete session records. History is kept in a LinkedList.
- **Automated eligibility check:** a live panel shows fee clearance, theory sessions and practical sessions for the selected student and a final decision (ELIGIBLE FOR TEST or NOT ELIGIBLE).
- **Test scheduling and outcomes:** a test can be scheduled only for an eligible student; outcomes can be updated to SCHEDULED, PASSED or FAILED; a test can be cancelled (deleted).
- **Search:** case-insensitive partial match on Student ID, name or course track.
- **Sorting and analytics:** students ranked by completed sessions using a TreeMap in descending order; tests listed in chronological order; full school summary with an individual eligibility audit.
- **Strict validation:** required fields, duplicate Student ID prevention, numeric and range checks on fees, strict calendar validation of dates.
- **Dark-themed GUI:** sidebar navigation with live counters, custom table renderers and colored status badges.

4. Java Concepts Used

Java concept	Application in the project
Classes and objects	Student, Course, TrainingSession, Test
Constructors	TwoWheelerCourse() and FourWheelerCourse() call super(...) with fee and required sessions; Student, TrainingSession and Test constructors initialize their fields
Encapsulation	Private fields with public getters and setters in all model classes
Abstract class	Course, with the abstract method getType()
Inheritance and polymorphism	TwoWheelerCourse and FourWheelerCourse extend Course and override getType(); code uses the Course reference type
ArrayList	studentList (enrollments) and testList (driving tests)
LinkedList	sessionHistory, the chronological training session log
HashMap	studentCourseMap: Student ID mapped to Course
TreeMap	TreeMap with Collections.reverseOrder(): completed-session count mapped to students, ranked in descending order

Java concept	Application in the project
CRUD	Enrollment, sessions and tests can be created, read, updated and deleted
Searching	Multi-attribute filter by ID, name or course track
Sorting	TreeMap ranking and Comparator.comparing(Test::getDate)
Swing GUI	JFrame, CardLayout, JTable with DefaultTableModel, JComboBox, JTextField, JTextArea, JOptionPane, custom cell renderer
Exception handling	Custom checked exceptions InvalidDataException and IneligibleException with try/catch
Validation	Session count, test eligibility, fees, dates, duplicate IDs
Lambdas and streams-style API	Action listeners as lambdas; removelf; computelfAbsent

5. Software Design Overview

The whole application lives in one source file, `DrivingSchoolApp.java`. The main class `DrivingSchoolApp` extends `JFrame` and holds the data collections, the GUI components and all event handling. The domain classes and exceptions are package-private classes declared below it in the same file.

Layout of the source file

Section	Contents
Main frame	Collections, color palette, constructor, look-and-feel setup
Sidebar navigation	<code>createSidebar</code> , <code>createNavButton</code> , <code>switchView</code> , <code>updateSidebarStats</code>
View 1 - Students	<code>createStudentsView</code> , <code>addStudent</code> , <code>updateStudent</code> , <code>deleteStudent</code> , <code>parseAndValidateFee</code> , <code>clearStudentForm</code>
View 2 - Sessions	<code>createSessionsView</code> (log, update, delete, clear handlers)
View 3 - Tests	<code>createTestsView</code> , <code>previewEligibility</code> , <code>checkEligibility</code> , <code>scheduleTest</code> , <code>parseAndValidateDate</code>
View 4 - Reports	<code>createReportsView</code> (search, <code>TreeMap</code> sort, date sort, full report)
UI helpers	<code>showDarkDialog</code> , <code>showDarkConfirm</code> , <code>createViewHeader</code> , <code>createCardPanel</code> , <code>createInput</code> , <code>createComboBox</code> , <code>createActionButton</code> , <code>createCustomTable</code> , <code>StatusBadgeRenderer</code>
Data helpers	<code>findStudent</code> , <code>refreshStudentTable</code> , <code>refreshSessionTable</code> , <code>refreshTestTable</code> , <code>refreshDropdowns</code> , <code>loadSampleData</code>
Domain models	<code>Student</code> , <code>Course</code> , <code>TwoWheelerCourse</code> , <code>FourWheelerCourse</code> , <code>TrainingSession</code> , <code>Test</code>
Exceptions	<code>InvalidDataException</code> , <code>IneligibleException</code>

Class relationships

```
Course (abstract)
|-- TwoWheelerCourse    fee Rs.3500, 5 theory, 10 practical
|-- FourWheelerCourse   fee Rs.7500, 8 theory, 15 practical
```

```
DrivingSchoolApp (extends JFrame)
|-- List<Student>        studentList      (ArrayList)
|-- List<TrainingSession> sessionHistory   (LinkedList)
|-- Map<String, Course>  studentCourseMap (HashMap)
|-- List<Test>           testList         (ArrayList)
```

`TrainingSession` and `Test` refer to a student by `studentId` (String).
`Student` is linked to its `Course` through `studentCourseMap` (Student ID key).

The `Student` class does not hold a `Course` object; the link between a student and a course is kept only in the `HashMap`, keyed by Student ID.

6. Class Descriptions

Student

Member	Details
Fields (private)	String id, name, phone; double feePaid
Constructor	Student(id, name, phone, feePaid)
Getters	getId, getName, getPhone, getFeePaid
Setters	setName, setPhone, setFeePaid (the ID cannot be changed)

Course (abstract), TwoWheelerCourse, FourWheelerCourse

Class	Fee (Rs.)	Required theory	Required practical	getType()
TwoWheelerCourse	3,500	5	10	Two-Wheeler
FourWheelerCourse	7,500	8	15	Four-Wheeler

Course holds private fields name, fee, requiredTheory and requiredPractical, with getters and the abstract method `getType()`. The subclasses pass their fixed values to the superclass constructor.

TrainingSession

Field	Meaning
sessionId	Generated as SESS-(history size + 101)
studentId	ID of the student the session belongs to
type	THEORY or PRACTICAL
date	YYYY-MM-DD string
status	COMPLETED, SCHEDULED or ABSENT

Getters exist for all fields; setters exist for type, date and status.

Test

Field	Meaning
testId	Generated as TEST-(test count + 501)
studentId	Candidate's Student ID
date	YYYY-MM-DD string
track	Two-Wheeler or Four-Wheeler (from <code>Course.getType()</code>)
result	SCHEDULED, PASSED or FAILED

Only setResult is provided as a setter, because the outcome is the only attribute that changes after scheduling.

7. Data Structures

Structure	Variable	Why it is used
ArrayList<Student>	studentList	Fast indexed access and iteration over enrollments
LinkedList<TrainingSession>	sessionHistory	Session history appended in the order it is logged; efficient removal while iterating
HashMap<String, Course>	studentCourseMap	Constant-time lookup of a student's course by ID
ArrayList<Test>	testList	Stores driving tests; copied and sorted for the date report
TreeMap<Integer, List<Student>>	local, in the ranking handler	Keys are completed-session counts in reverse order, so students with more completed sessions come first; students with equal counts share one list

8. Business Rules

8.1 Course track rules

Course track	Course fee	Required theory sessions	Required practical sessions
Two-Wheeler	Rs. 3,500	5	10
Four-Wheeler	Rs. 7,500	8	15

8.2 Test eligibility

A student is eligible for the final driving test only when all three conditions are true. Only sessions with status COMPLETED are counted; SCHEDULED and ABSENT sessions do not count.

Condition	Rule (checkEligibility)
Fee clearance	feePaid is greater than or equal to the course fee
Theory	Completed THEORY sessions is at least the required theory count
Practical	Completed PRACTICAL sessions is at least the required practical count

```
checkEligibility(studentId, log):
    student = findStudent(studentId); course = studentCourseMap.get(studentId)
    if student or course is missing -> return false
    pass = true
    if feePaid < course.fee          -> log "Fee: Pending", pass = false
    count COMPLETED THEORY and PRACTICAL sessions for this student
    if theory    < requiredTheory    -> log "Shortage", pass = false
    if practical < requiredPractical -> log "Shortage", pass = false
    return pass
```

The method also writes a line-by-line explanation into a StringBuilder, which is shown in the eligibility panel, in the error dialog when scheduling is blocked, and in the full school report.

8.3 Fee status

In the student table the status is PAID when feePaid is at least the course fee; otherwise it shows DUE with the remaining amount.

9. Exception Handling and Validation

Custom checked exceptions

Exception	Thrown when
InvalidDataException	Input is missing, malformed or out of range (fields, fee, date, selection)
IneligibleException	Scheduling a test for a student who does not meet the eligibility rules

Validation rules

Area	Rule
Enrollment	All four fields (ID, name, phone, fee) are required
Duplicate ID	A Student ID that already exists (case-insensitive) is rejected
Fee format	Must be numeric; otherwise a message shows an example amount
Fee range	Not negative; not above Rs. 30,000; not above the selected course fee
Update	A student must be selected; name and phone cannot be empty
Date format	Must match four digits, dash, two digits, dash, two digits (YYYY-MM-DD)
Calendar date	Parsed with a STRICT resolver, so impossible dates such as 2026-02-30 are rejected
Year range	Year must be between 2000 and 2100
Selections	A student must be chosen before logging a session or scheduling a test; a session or test must be selected before update/delete
Deletion	Student, session and test deletions ask for confirmation first

Errors are shown in dark-themed JOptionPane dialogs. A general catch of Exception in addStudent and updateStudent guards against unexpected errors.

10. Graphical User Interface

The window is 1240 x 800 pixels (minimum 1100 x 720) and uses a custom dark "zinc" color palette applied through UIManager defaults and custom components. The layout is a left sidebar plus a central panel that uses a `CardLayout` to switch between four views.

Area	Description
Sidebar	Application branding, four navigation buttons (Students, Sessions, Tests, Reports) and live counters for total students, sessions and tests
Students view	Form card (ID, name, phone, course track, fee paid) with Add, Update, Delete and Clear buttons; table of students with a fee status column; clicking a row fills the form
Sessions view	Form (student, type, status, date) with Log Session, Update, Delete and Clear; table of sessions with a colored status badge
Tests view	Form (candidate, test date, outcome) with Schedule Test, Update Outcome and Delete; a live eligibility text panel; table of tests
Reports view	Filter text box with Search, Sort by Sessions (TreeMap), Sort Tests by Date and Full School Report buttons; monospaced output area

Other GUI details

- The student and test dropdowns are refreshed after every change and display entries as "ID - Name".
- The eligibility panel updates automatically when a student is chosen, when sessions change, and when a test row is selected.
- `StatusBadgeRenderer` colors status text: green for PAID, COMPLETED and PASSED; amber for DUE and SCHEDULED; red for ABSENT and FAILED.
- The Swing interface is created on the event dispatch thread using `SwingUtilities.invokeLater` in `main()`.

11. Module Walkthrough

11.1 Student enrollment

Add Student validates the fields and fee, creates the correct Course subclass from the selected track, stores the Student in the ArrayList and the course in the HashMap, then refreshes the table, dropdowns and sidebar counters. Update changes name, phone, fee and course for the selected ID. Delete asks for confirmation and removes the student, the HashMap entry, and all sessions and tests belonging to that student.

11.2 Session scheduling and attendance

Log Session reads the chosen student, type, status and a validated date, creates a `TrainingSession` with the next generated ID and appends it to the LinkedList. After every log, update or delete, the eligibility panel and counters are refreshed.

11.3 Eligibility and test scheduling

Schedule Test validates the date, runs checkEligibility and, if the student is ineligible, throws IneligibleException so the dialog shows exactly which requirement failed. Otherwise a Test with status SCHEDULED is created using the course type as its track. Update Outcome changes the result of the selected test (or the student's latest test if no row is selected). Delete cancels a test after confirmation.

11.4 Search, sorting and reports

- **Search:** lowercase partial match against ID, name and course track; prints a table and a total match count.
- **Sort by Sessions (TreeMap):** counts COMPLETED sessions per student, groups students by count in a descending TreeMap and prints each group.
- **Sort Tests by Date:** copies testList and sorts it with Comparator.comparing(Test::getDate); the YYYY-MM-DD format sorts correctly as text.
- **Full School Report:** totals for students, sessions and tests, then each student's eligibility breakdown with the status READY FOR FINAL TEST or INELIGIBLE.

12. Sample Data

loadSampleData() preloads the application with 8 students, their sessions and 4 tests so every feature can be demonstrated immediately.

Student ID	Name	Track	Fee paid (Rs.)
STU-101	Aarav Sharma	Two-Wheeler	3,500
STU-102	Ananya Verma	Four-Wheeler	4,000
STU-103	Rohan Mehta	Four-Wheeler	7,500
STU-104	Priya Patel	Two-Wheeler	3,500
STU-105	Vikram Singh	Four-Wheeler	7,500
STU-106	Sneha Kulkarni	Two-Wheeler	3,500
STU-107	Kabir Deshmukh	Four-Wheeler	7,500
STU-108	Neha Joshi	Two-Wheeler	3,500

Test ID	Student	Date	Track	Result
TEST-501	STU-101	2026-04-10	Two-Wheeler	SCHEDULED
TEST-502	STU-104	2026-04-12	Two-Wheeler	SCHEDULED
TEST-503	STU-105	2026-04-05	Four-Wheeler	PASSED
TEST-504	STU-108	2026-03-28	Two-Wheeler	FAILED

13. How to Compile and Run

Prerequisite: Java Development Kit (JDK) 11 or higher.

```
git clone https://github.com/kartikwagh21/Java_Major_Project.git
cd Java_Major_Project
```

```
javac DrivingSchoolApp.java
java DrivingSchoolApp
```

```
# Java 11+ can also run the source file directly:
java DrivingSchoolApp.java
```

The application needs no external libraries and no database; it uses only the standard Java library (java.awt, javax.swing, java.time, java.util).

14. Test Scenarios and Expected Results

The results below follow from the rules in the code applied to the preloaded sample data.

Student	Fee	Theory	Practical	Eligibility result
STU-101 (2W)	Paid in full	5/5	10/10	ELIGIBLE (one theory session was ABSENT but five were completed)

Student	Fee	Theory	Practical	Eligibility result
STU-102 (4W)	Pending (4,000 of 7,500)	8/8	15/15	NOT ELIGIBLE (fee shortage)
STU-103 (4W)	Paid in full	2/8	0/15	NOT ELIGIBLE (session shortage)
STU-104 (2W)	Paid in full	5/5	10/10	ELIGIBLE
STU-105 (4W)	Paid in full	8/8	15/15	ELIGIBLE
STU-106 to STU-108	Paid in full	0	0	NOT ELIGIBLE (no completed sessions)

Ranking by completed sessions (TreeMap, descending)

Completed sessions	Students
23	STU-102, STU-105
15	STU-101, STU-104
2	STU-103
0	STU-106, STU-107, STU-108

Other test cases to try

Action	Expected result
Add a student with an existing ID	Error: ID is already enrolled
Enter fee "abc"	Error: invalid fee format
Enter fee 9000 for Two-Wheeler	Error: fee exceeds course fee
Log a session dated 2026-02-30	Error: invalid calendar date
Log a session dated 30-03-2026	Error: use YYYY-MM-DD
Schedule a test for STU-103	Blocked: IneligibleException lists the shortages
Schedule a test for STU-102 after paying 7,500	Allowed once the fee equals the course fee
Sort Tests by Date	Order: TEST-504, TEST-503, TEST-501, TEST-502
Search "four"	Returns all Four-Wheeler students
Delete a student	Student, sessions and tests removed; counters updated

15. Output Screenshots

The screenshots below were taken from the running application. They show the preloaded sample data, with one change made during the demonstration: the fee paid for STU-105 (Vikram Singh) was updated from Rs. 7,500 to Rs. 3,000. Because of this, STU-105 appears as DUE and INELIGIBLE in the later screenshots, which differs from the default sample data in Section 12.

Figure 1: Student Enrollment view

DRIVING SCHOOL
Management Portal

Student Enrollment
Manage student registrations and course track assignments (CRUD)

Student ID

Full Name

Phone Number

Course Track
Two-Wheeler

Fee Paid (₹)

Buttons: Add Student, Update, Delete, Clear

ID	Name	Phone	Track	Fee Paid	Status
STU-101	Aarav Sharma	9876543210	Two-Wheeler	₹3500.0	PAID
STU-102	Ananya Verma	9123456789	Four-Wheeler	₹4000.0	DUE (₹3500.0)
STU-103	Rohan Mehta	9988776655	Four-Wheeler	₹7500.0	PAID
STU-104	Priya Patel	9811223344	Two-Wheeler	₹3500.0	PAID
STU-105	Vikram Singh	9765432109	Four-Wheeler	₹7500.0	PAID
STU-106	Sneha Kulkarni	9654321870	Two-Wheeler	₹3500.0	PAID
STU-107	Kabir Deshmukh	9543216789	Four-Wheeler	₹7500.0	PAID
STU-108	Neha Joshi	9432165870	Two-Wheeler	₹3500.0	PAID

Students: 8
Sessions Logged: 80
Tests Scheduled: 4

Student form with Add, Update, Delete and Clear buttons, and the enrollment table with PAID / DUE fee status. STU-102 shows DUE (Rs. 3500.0). The sidebar counters show 8 students, 80 sessions and 4 tests.

Figure 2: Updating a student record

DRIVING SCHOOL

Management Portal

Student Enrollment

Training Sessions

Test & Eligibility

Search & Analytics

Students: 8
Sessions Logged: 80
Tests Scheduled: 4

Student Enrollment

Manage student registrations and course track assignments (CRUD)

Student ID

STU-105

Full Name

Vikram Singh

Phone Number

9765432109

Course Track

Four-Wheeler

Fee Paid (₹)

3000

Add Student

Update

Delete

Clear

ID	Name	Phone	Track	Fee Paid	Status
STU-101	Aarav Sharma	9876543210	Two-Wheeler	₹3500.0	PAID
STU-102	Ananya Verma	9123456789	Four-Wheeler	₹4000.0	DUE (₹3500.0)
STU-103	Rohan Mehta	9988776655	Four-Wheeler	₹7500.0	PAID
STU-104	Priya Patel	9811223344	Two-Wheeler	₹3500.0	PAID
STU-105	Vikram Singh	9765432109	Four-Wheeler	₹3000.0	DUE (₹4500.0)
STU-106	Sneha Kulkarni	9654321870	Two-Wheeler	₹3500.0	PAID
STU-107	Kabir Deshmukh	9543216789	Four-Wheeler	₹7500.0	PAID
STU-108	Neha Joshi	9432165870	Two-Wheeler	₹3500.0	PAID

Success

Student details updated successfully.

OK

STU-105 selected and updated (fee changed to 3000). The success dialog confirms the update and the table now shows DUE (Rs. 4500.0) for this student.

Figure 3: Training Sessions and Attendance view

DRIVING SCHOOL
Management Portal

Student Enrollment

Training Sessions

Test & Eligibility

Search & Analytics

Students: 8
Sessions Logged: 80
Tests Scheduled: 4

Training Sessions & Attendance
Log theory and practical sessions into LinkedList history log

Select Student
STU-101 - Aarav Sharma

Session Type
PRACTICAL

Attendance Status
COMPLETED

Session Date (YYYY-MM-DD)
2026-03-16

Log Session

Update

Delete

Clear

Session ID	Student ID	Type	Date	Status
SESS-101	STU-101	THEORY	2026-03-01	COMPLETED
SESS-102	STU-101	THEORY	2026-03-02	COMPLETED
SESS-103	STU-101	THEORY	2026-03-03	ABSENT
SESS-104	STU-101	THEORY	2026-03-04	COMPLETED
SESS-105	STU-101	THEORY	2026-03-05	COMPLETED
SESS-106	STU-101	THEORY	2026-03-06	COMPLETED
SESS-111	STU-101	PRACTICAL	2026-03-11	COMPLETED
SESS-112	STU-101	PRACTICAL	2026-03-12	COMPLETED
SESS-113	STU-101	PRACTICAL	2026-03-13	COMPLETED
SESS-114	STU-101	PRACTICAL	2026-03-14	COMPLETED
SESS-115	STU-101	PRACTICAL	2026-03-15	COMPLETED
SESS-116	STU-101	PRACTICAL	2026-03-16	COMPLETED
SESS-117	STU-101	PRACTICAL	2026-03-17	COMPLETED
SESS-118	STU-101	PRACTICAL	2026-03-18	COMPLETED
SESS-119	STU-101	PRACTICAL	2026-03-19	COMPLETED
SESS-120	STU-101	PRACTICAL	2026-03-20	COMPLETED
SESS-131	STU-102	THEORY	2026-03-01	COMPLETED
SESS-132	STU-102	THEORY	2026-03-02	COMPLETED
SESS-133	STU-102	THEORY	2026-03-03	COMPLETED
SESS-134	STU-102	THEORY	2026-03-04	COMPLETED
SESS-135	STU-102	THEORY	2026-03-05	COMPLETED
SESS-136	STU-102	THEORY	2026-03-06	COMPLETED
SESS-137	STU-102	THEORY	2026-03-07	COMPLETED
SESS-138	STU-102	THEORY	2026-03-08	COMPLETED
SESS-141	STU-102	PRACTICAL	2026-03-11	COMPLETED

Session form (student, type, attendance status, date) with Log Session, Update, Delete and Clear. The table lists the LinkedList session history with colored status; SESS-103 is ABSENT.

Figure 4: Test Eligibility and Scheduling view

DRIVING SCHOOL

Management Portal

Student Enrollment

Training Sessions

Test & Eligibility

Search & Analytics

Students: 8
Sessions Logged: 80
Tests Scheduled: 4

Test Eligibility & Scheduling

Verify session/fee requirements and schedule final driving tests

Candidate Student

STU-104 - Priya Patel

Test Date (YYYY-MM-DD)

2026-04-12

Test Outcome

SCHEDULED

Eligibility Status Preview

- Fee: Paid in full (₹3500.0)
- Theory: 5/5 (Satisfied)
- Practical: 10/10 (Satisfied)

Final Decision: ELIGIBLE FOR TEST

Schedule Te...

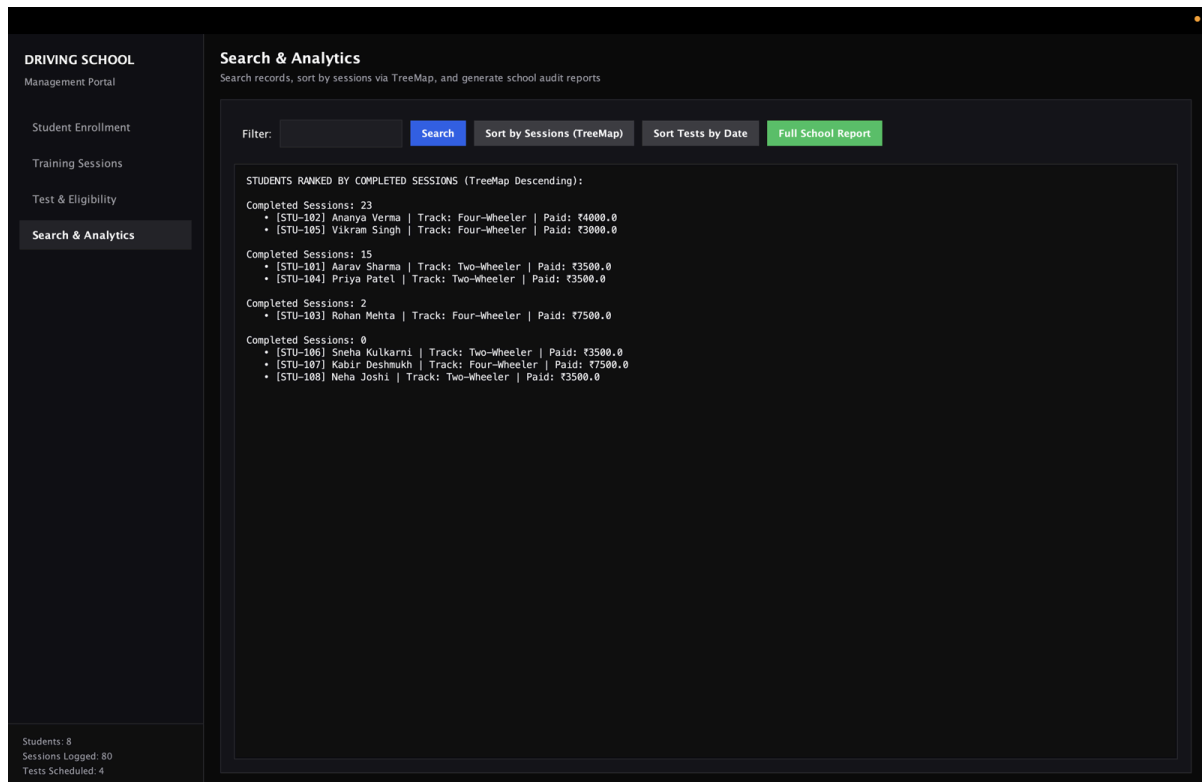
Update Out...

Cancel / De...

Test ID	Student ID	Track	Test Date	Outcome
TEST-501	STU-101	Two-Wheeler	2026-04-10	SCHEDULED
TEST-502	STU-104	Two-Wheeler	2026-04-12	SCHEDULED
TEST-503	STU-105	Four-Wheeler	2026-04-05	PASSED
TEST-504	STU-108	Two-Wheeler	2026-03-28	FAILED

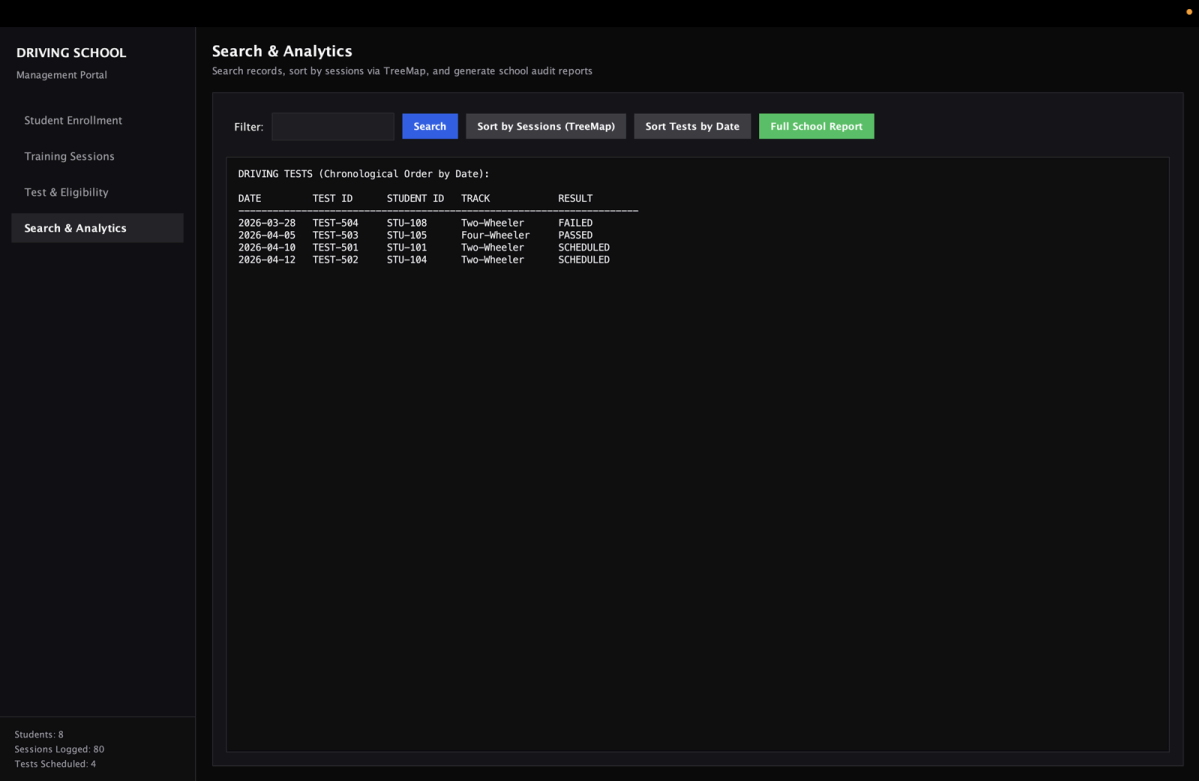
Live eligibility preview for STU-104 (Priya Patel): fee paid in full, theory 5/5, practical 10/10, final decision ELIGIBLE FOR TEST. The table lists tests with outcomes SCHEDULED, PASSED and FAILED.

Figure 5: Students ranked by completed sessions (TreeMap)



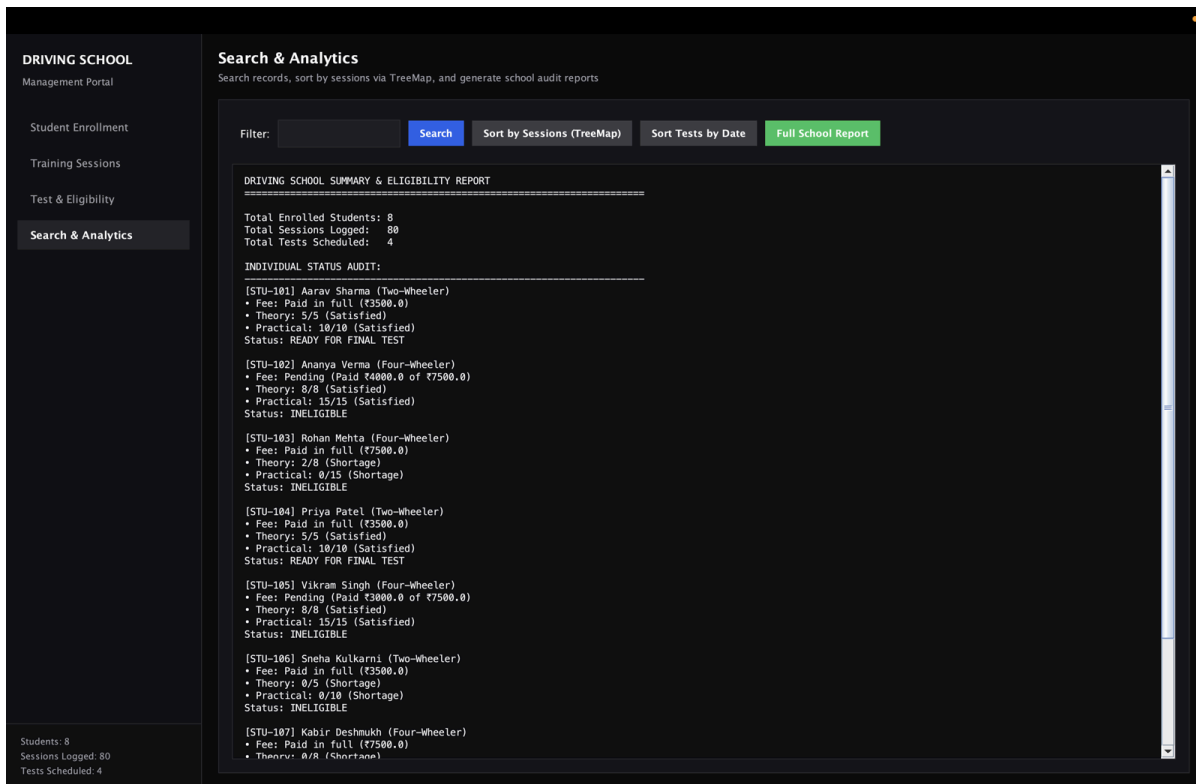
Descending TreeMap output: 23 sessions (STU-102, STU-105), 15 (STU-101, STU-104), 2 (STU-103) and 0 (STU-106, STU-107, STU-108).

Figure 6: Driving tests sorted by date



Chronological order: TEST-504, TEST-503, TEST-501, TEST-502.

Figure 7: Full School Report



Summary totals and an individual eligibility audit for every student, with READY FOR FINAL TEST or INELIGIBLE. STU-102 and STU-105 are ineligible because of pending fees; STU-103 and STU-106 because of session shortages.

16. Limitations and Future Enhancements

Observations on the current version

- Data is held in memory only, so all changes are lost when the application is closed; the sample data is reloaded on each start.
- Session and test IDs are generated from the current list size. After deleting a record, a new record may receive an ID that already exists.
- A student can be given more than one test, and the test table does not prevent duplicates for the same student.
- Changing a student's course track in Update replaces the course object but does not re-check existing sessions.
- Some sample tests (for example STU-108) were added directly and were not passed through the eligibility check.

Possible enhancements

- Save and load data with files (serialization or CSV) or a database through JDBC.
- Generate unique IDs with a counter or UUID.
- Split the single file into packages (model, service, ui) following an MVC structure.
- Add instructor and vehicle management, and a calendar view for sessions.
- Export reports to PDF or CSV.
- Add user login with roles (admin and receptionist).

17. Conclusion

The Driving School Enrollment & Test Management System covers every requirement of Case Study 90. It uses an abstract Course class with two subclasses to separate the two training tracks, ArrayList, LinkedList, HashMap and TreeMap for the required storage and ranking tasks, custom checked exceptions with thorough validation, and a complete Swing interface covering enrollment, attendance, eligibility checking, test scheduling, search, sorting and reporting.